

Lab 7

ELECENG 3EY4

Alaa Abdelsalam – abdel74 – 400396373

Tina Ismail – ismait1 – 400299939

Marisa Patel – patem156 – 400393635

March 21, 2024

Table of Contents

Contribution	2
Activity 1	2
Activity 2	3
Activity 3	4
Activity 4	5
Activity 5	5
Activity 6	6
Activity 7	7
Activity 8	8
Activity 9	8
Activity 10	9
Activity 11	11
Activity 12	11
Activity 13	12
Activity 14	12
Activity 15	12
Activity 16	13
Activity 17	15
Activity 18	15
Activity 19	16
Activity 20	16
Activity 21	17
Activity 22	17
Activity 23	18
Activity 24	19
Activity 25	20

Contribution

Activities 1 to 25: Alaa, Tina, Marisa

Activity 1

Derivation of Homogenous Transformations that Relate the Frame “base_link” to the Frames “laser,” “imu,” and “camera,” i.e., $T_{laser}^{base_link}$, $T_{imu}^{base_link}$, $T_{camera}^{base_link}$:

Derivation for “base_link” to “laser”:

$$R = 180^\circ \text{ (rotate } 180^\circ \text{)}$$

$$T = 12.86x + 76z \text{ (in mm)}$$

$$T_{laser}^{base_link} = \begin{bmatrix} \cos(180) & -\sin(180) & 0 & 12.86 \\ \sin(180) & \cos(180) & 0 & 0 \\ 0 & 0 & 0 & 76 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$T_{laser}^{base_link} = \begin{bmatrix} -1 & 0 & 0 & 0.01286 \\ 0 & -1 & 0 & 0 \\ 0 & 0 & 1 & 0.076 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$\alpha = 180^\circ = \pi$$

$$q = q_w + q_{xi} + q_{yi} + q_{zk}$$

$$q = \cos \frac{\alpha}{2} + \sin \frac{\alpha}{2} [xi + yi + zk] = 0 + 1k$$

$$base_link_to_laser = [0.01286 \ 0 \ 0.076 \ 0 \ 0 \ 1 \ 0]$$

Derivation for “base_link” to “imu”:

$R = 90z$ (*rotate 90°*)

$T = 12.86x + 61.19z$ (*in mm*)

$$T_{imu}^{base_link} = \begin{bmatrix} \cos(90) & -\sin(90) & 0 & 12.86 \\ \sin(90) & \cos(90) & 0 & 0 \\ 0 & 0 & 1 & 61.19 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$T_{imu}^{base_link} = \begin{bmatrix} 0 & -1 & 0 & 0.01286 \\ -1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0.0619 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$\alpha = 90^\circ = \frac{\pi}{2}$$

$$q = q_w + q_{xi} + q_{yi} + q_{zk}$$

$$q = \cos \frac{\alpha}{2} + \sin \frac{\alpha}{2} [xi + yi + zk] = \frac{\sqrt{2}}{2} + \frac{\sqrt{2}}{2} k$$

$$base_link_to_imu = [0.01286 \ 0 \ 0.0619 \ 0 \ 0 \ 0.707 \ 0.707]$$

Derivation for “base_link” to “camera”:

$R = 90z$ (*rotate 90°, flip to the side*)

$T = 283.03x + 86y$ (*in mm*)

Activity 2

In activity 2, we completed the ROS static transformation in the “experiment.launch” file that was provided on Avenue.

Calculated Values based on Figures 1 and 2 from the Lab Manual:

$$static_transform_publisher = [x \ y \ z \ qx \ qy \ qz \ qw]$$

From the schematic MacAEV figure in the lab manual, the dimension of the imu are:

$$x = 12.86\text{mm}, y = 0\text{mm}, z = 55 + 6.19 = 61.19\text{mm}$$

$$base_link_to_imu = [0.01286 \ 0 \ 0.0619 \ 0 \ 0 \ 0.707 \ 0.707]$$

The dimension of the laser is:

$x = 12.86\text{mm}$, $y = 0\text{mm}$, $z = 55 + 21 = 76\text{mm}$

base_link_to_laser = [0.01286 0 0.076 0 0 1 0]

Completed ROS Static Transformation Code:

```
<node pkg="tf2_ros" type="static_transform_publisher"
name="base_link_to_imu"
  args="0.01286 0 0.0619 0 0 0.707 0.707 base_link imu" />
<node pkg="tf2_ros" type="static_transform_publisher"
name="base_link_to_laser"
  args="0.01286 0 0.076 0 0 1 0 base_link laser" />
```

The purpose of the “base_link_to_imu” transformation is to specify the orientation and position of the IMU sensor relative to the “base_link” frame of the vehicle. This transformation defines the relationship between the “base_link” and the “imu” frames. This provides significant information regarding sensor data, orientation, and motion of the vehicle. The purpose of the “base_link_to_laser” transformation is to specify the position of the laser sensor relative to the “base_link” frame. This transformation defines the relationship between the “base_link” and the “laser” frames. It ensures that the sensor data can be accurately interpreted. This is essential for mapping, localization, navigation, and obstacle detection, which means that the specific orientation and position of the laser sensor is known while it is on the vehicle.

Activity 3

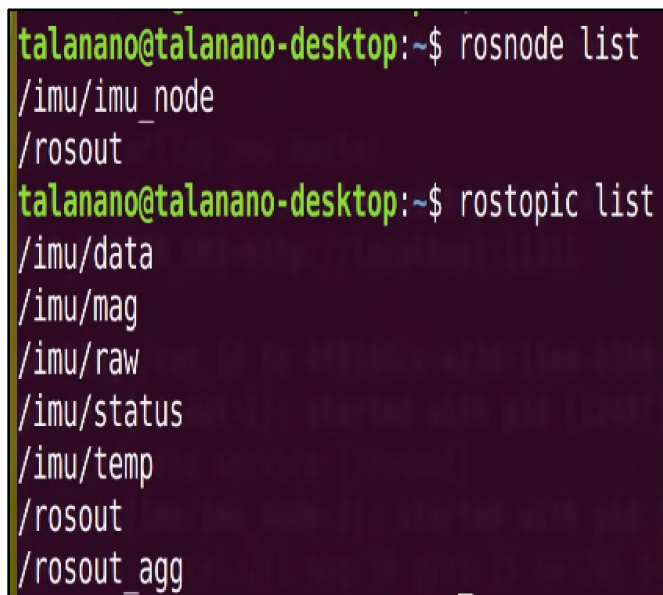
The orientation of a frame is expressed in a quaternion vector. Quaternion vectors are used to represent the orientation of a frame in a 3D space. A quaternion is a vector with four dimensions including x, y, z, and w. This represents the rotation of a frame with respect to a reference frame. It also represents the orientation of a frame by identifying the rotation that is needed to align it with the reference frame. The four dimensions of a quaternion each represent a part of the quaternion vector. The x, y, and z components of the quaternion vector are used to define the axis of rotation. These specify the direction of the rotation axis. The w component of the quaternion vector represents the scalar portion of the quaternion. This defines the amount of rotation around the rotation axis. This is how the orientation of a frame is expressed in quaternion vectors.

Activity 4

In activity 4, we installed the IMU driver from GitHub so we can use an open-source ROS driver to read the IMU data over the 12C bus. To install the driver, we went into the “src” folder in the catkin workspace, and then cloned the GitHub repository using the “cd ~/catkin_ws/src” and “sudo git clone https://github.com/dheera/ros-imu-bno055” commands. Following this, we returned to the root directory and built a new package in the “src” folder using the “cd ~/catkin_ws” and “catkin_make” commands. Once we installed the driver, we examined the driver launch file contents and verified that it matched the contents provided in the lab manual.

Activity 5

In activity 5, we ran the launch file from the command line and obtained the list of published ROS topics related to the IMU. To obtain this list, we used the “rostopic list” command. In Figure 1 below, you can see the various topics that have been published related to the IMU. Two of the topics we looked at includes the “/imu/data” and “/imu/status” topics. The “/imu/status” topic provides the calibration status of the IMU. The original “Calibration status” value before calibrating was 51.

A terminal window with a dark purple background and green text. The prompt is 'talanano@talanano-desktop:~\$'. The first command is 'roscd list', which outputs '/imu/imu_node' and '/rosout'. The second command is 'rostopic list', which outputs a list of topics: '/imu/data', '/imu/mag', '/imu/raw', '/imu/status', '/imu/temp', '/rosout', and '/rosout_agg'.

```
talanano@talanano-desktop:~$ roscd list
/imu/imu_node
/rosout
talanano@talanano-desktop:~$ rostopic list
/imu/data
/imu/mag
/imu/raw
/imu/status
/imu/temp
/rosout
/rosout_agg
```

Figure 1. “rostopic list” Command and Output

Activity 6

To calibrate the IMU, we followed the calibration procedure outlined on page 51 of the sensor data sheet. To calibrate the IMU, all three sensors including the accelerometer, gyroscope, and magnetometer need to be calibrated. We echoed the value of “Calibration status” on the terminal screen as we manually moved the vehicle. To calibrate the accelerometer, we placed the vehicle in six different stable position for a few seconds. To calibrate the gyroscope, we placed the vehicle in a single stable position for a period of a few seconds. To calibrate the magnetometer, we moved the vehicle in random movements. The “Calibration status” value we obtained after calibrating the IMU was 255, which is 11111111 in binary.

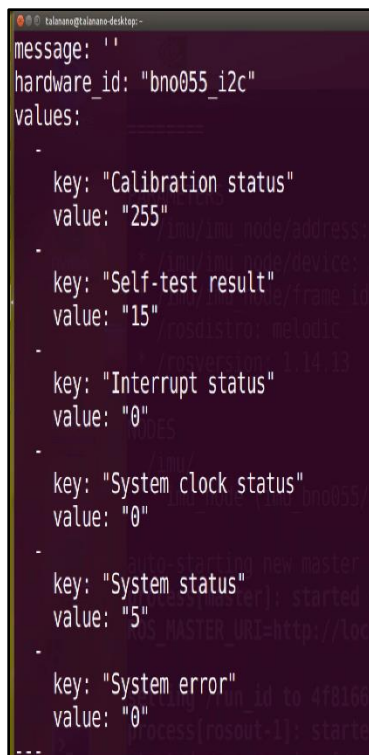
SYS Calibration Status: 11

GYR Calibration Status: 11

ACC Calibration Status: 11

MAG Calibration Status: 11

This means that the current system, Gyroscope, Accelerometer, and Magnetometer calibration status all read a value of 3, which indicated that they are all fully calibrated. According to this, the status value that would indicate a fully calibrated IMU is 255, which is 11111111 in binary. This is the same “Calibration status” value we obtained after calibration.

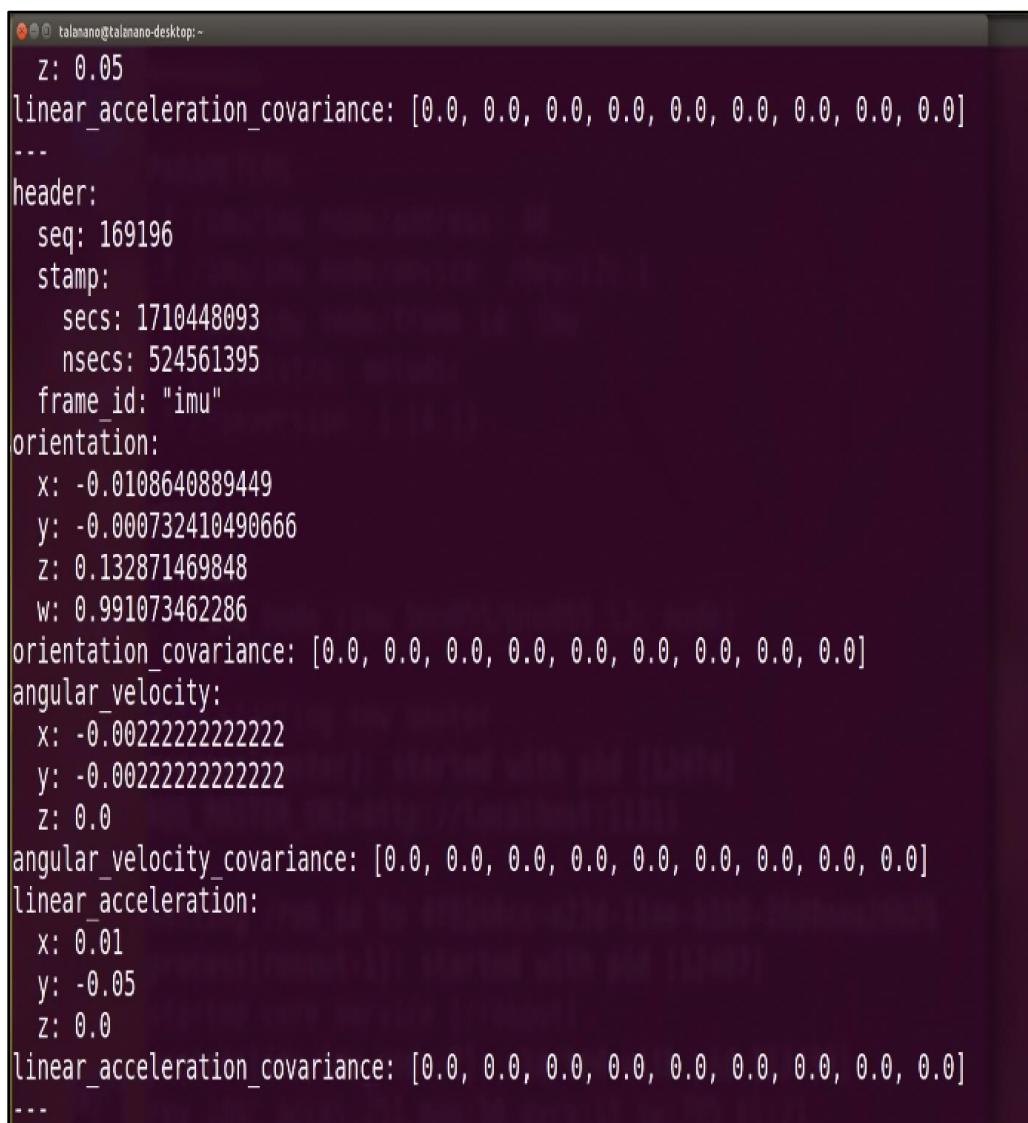
A terminal window with a dark background and light green text. The text shows a message structure with several key-value pairs. The 'Calibration status' key has a value of '255'. Other keys include 'Self-test result' (15), 'Interrupt status' (0), 'System clock status' (0), 'System status' (5), and 'System error' (0).

```
message: ''
hardware_id: "bno055_i2c"
values:
-
  key: "Calibration status"
  value: "255"
-
  key: "Self-test result"
  value: "15"
-
  key: "Interrupt status"
  value: "0"
-
  key: "System clock status"
  value: "0"
-
  key: "System status"
  value: "5"
-
  key: "System error"
  value: "0"
...
```

Figure 2. “Calibration status” Value after Calibrating the IMU

Activity 7

In activity 7, we confirmed that the IMU reported orientation is consistent with the convention of a fixed earth-attached frame with axes, x (East), y (North), and z (up). To do this, we aligned the y-axis of the IMU frame with the direction of the earth north and read its orientation value. Using Figure 1 from the lab manual, we can see that the IMU sensor is pointing towards the back of the car, so in order to align the sensor in the direction of the earth north, we placed the front of the car in the direction of the earth south so that back of the car is in the direction of the earth north. Figure 3 below shows that x, y, and z have an orientation value of approximately 0, and w is approximately 1. The reason why w is approximately 1 is because $\cos(90^\circ) = 1$. We can expect that x, y, and z are approximately 0 based on the derived equations from Activity 1.

A terminal window with a dark purple background and white text. The window title is 'talanano@talanano-desktop: ~'. The output shows various IMU sensor data fields. The 'orientation' section contains the values for x, y, z, and w. The 'linear_acceleration' section contains values for x, y, and z. The 'angular_velocity' section contains values for x, y, and z. The 'orientation_covariance' and 'angular_velocity_covariance' sections both show a list of nine zeros. The 'linear_acceleration_covariance' section also shows a list of nine zeros. The output is separated into sections by '---' markers.

```
talanano@talanano-desktop: ~
z: 0.05
linear_acceleration_covariance: [0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0]
---
header:
  seq: 169196
  stamp:
    secs: 1710448093
    nsecs: 524561395
  frame_id: "imu"
orientation:
  x: -0.0108640889449
  y: -0.000732410490666
  z: 0.132871469848
  w: 0.991073462286
orientation_covariance: [0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0]
angular_velocity:
  x: -0.00222222222222
  y: -0.00222222222222
  z: 0.0
angular_velocity_covariance: [0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0]
linear_acceleration:
  x: 0.01
  y: -0.05
  z: 0.0
linear_acceleration_covariance: [0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0]
---
```

Figure 3. Orientation Values for x, y, z, and w

Activity 8

In activity 8, we created a ROS package named “imu_tf_pkg” in the catkin workspace with the “roscpp,” “std_msgs,” “sensor_msgs,” and “tf” dependencies. Then, we added the “imu_tf_pub.cpp” file as a node to the ROS package. Then, we added the “imu_tf_pub.launch” file to the “launch” folder in the package. We also replaced the “CMakeLists.txt” file in the package with the “CMakeLists.txt” from Avenue. Finally, we built the package. The content in the “imu_tf_pub.cpp” file, which is now a node, subscribes to the IMU data that is published on a specific topic, which is “/imu/data.” After it receives the IMU message, it extracts the orientation data and converts it into a quaternion. Once this is done, it creates a “tf::StampedTransform” object, which represents the transformation between the “base_stabilized_frame” and the “base_frame” frames. The transformation done is based on the orientation data obtained from the IMU message. This transform is then broadcasted using “tf::TransformBroadcaster,” which makes the orientation data available to alternate parts of the ROS system. Therefore, the node converts the extracted IMU data into a transform, which is broadcasted to other nodes so those nodes can use the orientation data for different purposes such as navigation or visualization.

Activity 9

In activity 9, we launched the new node and verified that a transformation between the fixed frame “map” and the IMU frame “imu” was published over the topic “/tf.” To ensure that the IMU data is being published, we ensured that we first ran the “imu.launch” file from the “imu-bno055” package. Then, we launched the ROS visualization tool “rviz” and added the TF displays. We set the fixed frame to “map” and manually moved the vehicle. Figure 4 below shows the alignment of the “map” and “imu” frames in the ROS visualization tool “rviz.” The “map” frame is fixed with the axes x (East), y (North) and z (up).

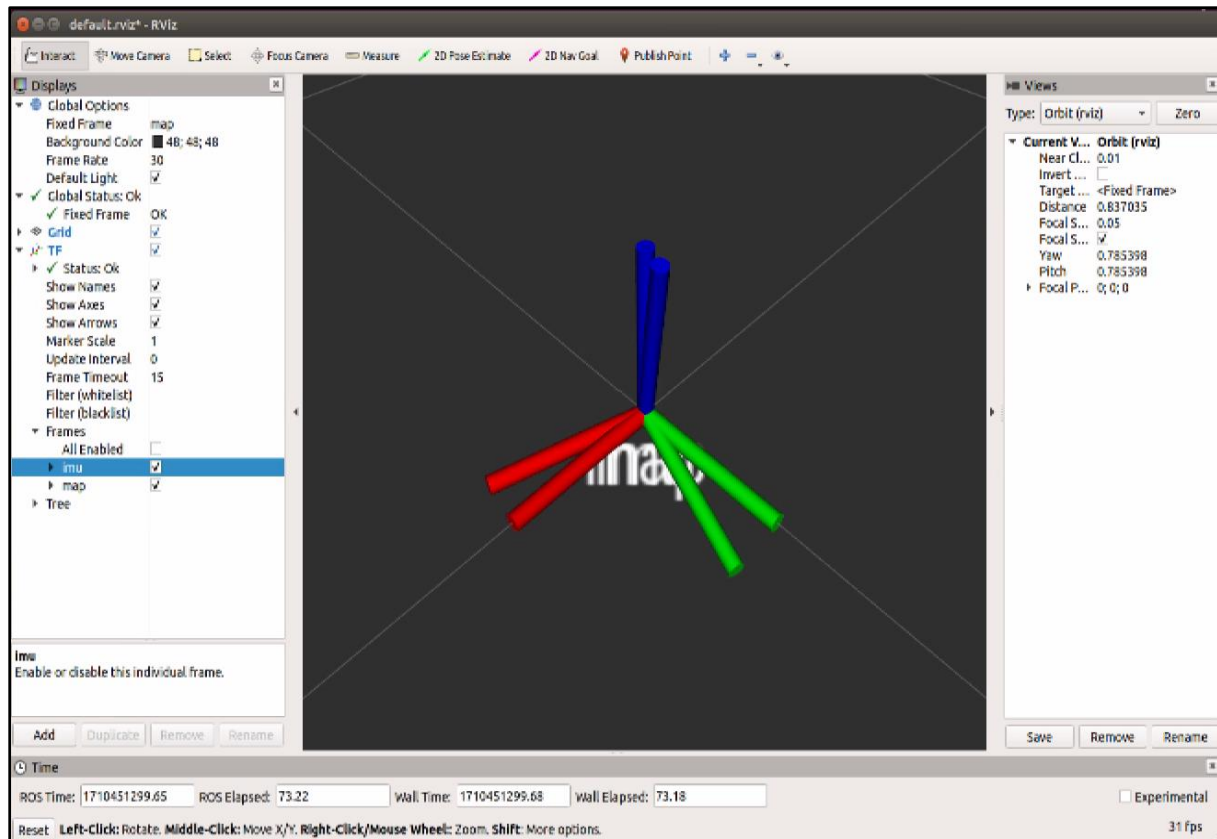


Figure 4. “rviz” showing the Alignment of “map” and “imu” Frames

Activity 10

In activity 10, we replaced the contents of the “experiment.cpp” file from Lab 6 with the contents of the “experiment_incomplete.cpp” file from Avenue. We completed the incomplete code lines in this file.

For the current angular velocity, we completed this line of code:

```
current_angular_velocity = [current_speed / wheelbase] * tan (current_steering_angle);
```

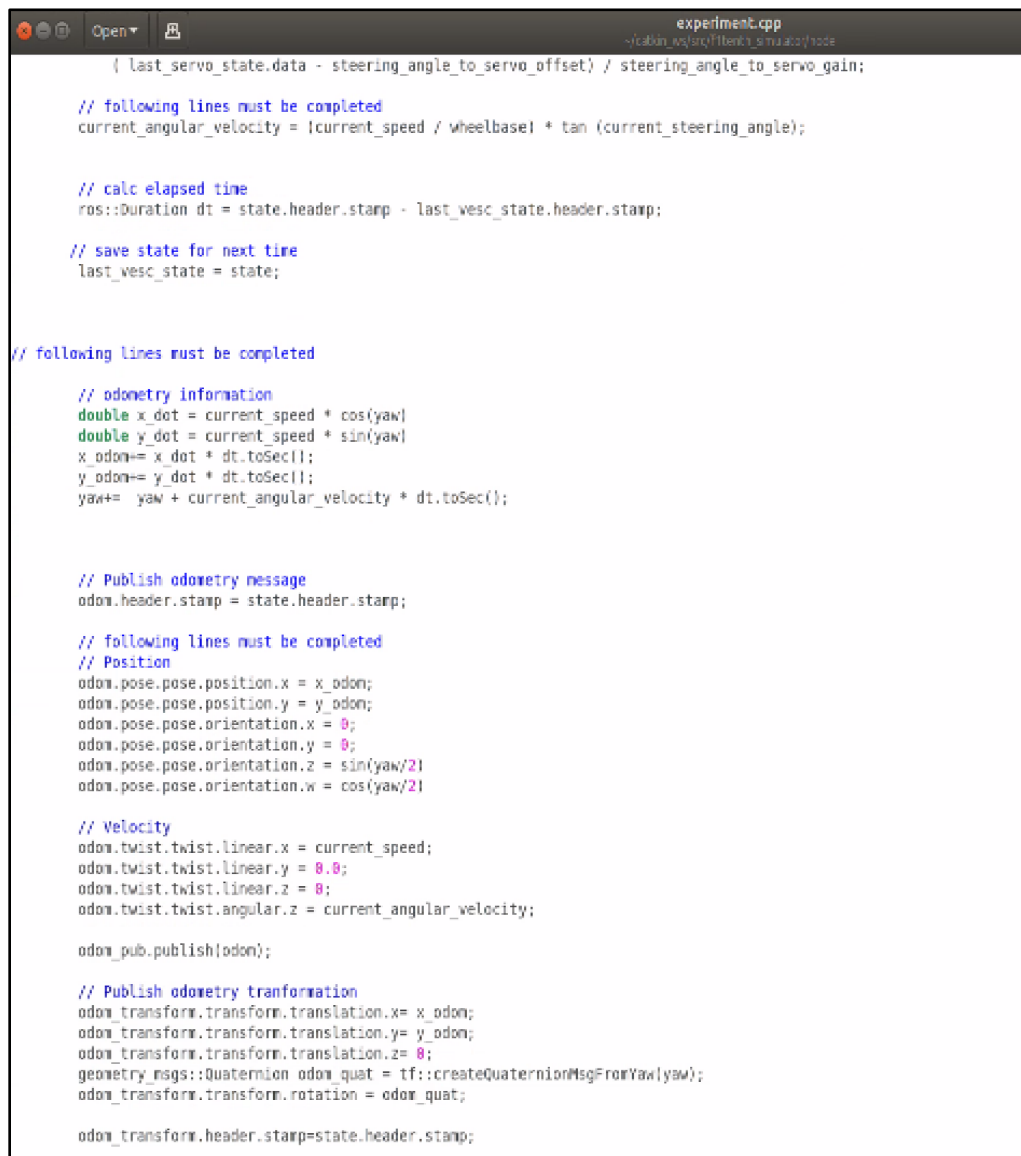
For the odometry information, we used equations 2 and 3 from the lab manual for “x_dot” and “y_dot,” which are $\dot{x} = v_s \cos \theta$ and $\dot{y} = v_s \sin \theta$. We used equations 8 and 9 from the lab manual for “x_dom” and “y_dom,” which are $x_{k+1} = x_k + \delta t_k v_{sk} \cos(\theta_k)$ and $y_{k+1} = y_k + \delta t_k v_{sk} \sin(\theta_k)$. We edited these lines of code:

```
double x_dot = current_speed * cos(yaw);
double y_dot = current_speed * sin(yaw);
x_odom += x_dot * dt.toSec();
y_odom += y_dot * dt.toSec();
yaw += yaw + current_angular_velocity * dt.toSec();
```

For the position information, we edited these lines of code:

```
odom.pose.pose.position.x = x_odom;  
odom.pose.pose.position.y = y_odom;  
odom.pose.pose.orientation.x = 0;  
odom.pose.pose.orientation.y = 0;  
odom.pose.pose.orientation.z = sin(yaw/2);  
odom.pose.pose.orientation.w = cos(yaw/2);
```

The modified node is used to implement the discrete-time integration of the wheel odometry. These odometry results are then published as a ROS topic and a ROS transformation.



```
experiment.cpp  
~/catkin_ws/src/Turtle3/simulation/node  
{ last_servo_state.data - steering_angle_to_servo_offset} / steering_angle_to_servo_gain;  
  
// following lines must be completed  
current_angular_velocity = (current_speed / wheelbase) * tan (current_steering_angle);  
  
// calc elapsed time  
ros::Duration dt = state.header.stamp - last_wesc_state.header.stamp;  
  
// save state for next time  
last_wesc_state = state;  
  
// following lines must be completed  
  
// odometry information  
double x_dot = current_speed * cos(yaw);  
double y_dot = current_speed * sin(yaw);  
x_odom+= x_dot * dt.toSec();  
y_odom+= y_dot * dt.toSec();  
yaw+= yaw + current_angular_velocity * dt.toSec();  
  
// Publish odometry message  
odom.header.stamp = state.header.stamp;  
  
// following lines must be completed  
// Position  
odom.pose.pose.position.x = x_odom;  
odom.pose.pose.position.y = y_odom;  
odom.pose.pose.orientation.x = 0;  
odom.pose.pose.orientation.y = 0;  
odom.pose.pose.orientation.z = sin(yaw/2);  
odom.pose.pose.orientation.w = cos(yaw/2);  
  
// Velocity  
odom.twist.twist.linear.x = current_speed;  
odom.twist.twist.linear.y = 0.0;  
odom.twist.twist.linear.z = 0;  
odom.twist.twist.angular.z = current_angular_velocity;  
  
odom_pub.publish(odom);  
  
// Publish odometry transformation  
odom_transform.transform.translation.x = x_odom;  
odom_transform.transform.translation.y = y_odom;  
odom_transform.transform.translation.z = 0;  
geometry_msgs::Quaternion odom_quat = tf::createQuaternionMsgFromYaw(yaw);  
odom_transform.transform.rotation = odom_quat;  
  
odom_transform.header.stamp=state.header.stamp;
```

Figure 5. “experiment.cpp” File with Completed Lines of Code

Activity 11

In activity 11, we made the required changes in the “experiment.cpp” file. Then, we changed the directory to the root of the catkin workspace and rebuilt the package. Finally, we launched the vehicle control code. To do this we used the “catkin_make,” “roscd fltenth_simulator,” and “roslaunch fltenth_simulator experiment.launch” commands.

Activity 12

In activity 12, we ran the ROS visualization tool “rviz,” set the fixed frame to “odom,” and added a TF display. We selected the “odom” and “base_link” frames for display. The two frames are initially aligned before starting to drive the vehicle, which can be seen in Figure 6 below. Using the joystick, we manually drove the vehicle and observed its configuration in the “rviz” visualization tool. Then, we attempted to re-position the vehicle back to its original configuration. Ideally, the two frames should have re-aligned, but this was not the case in practice. In Figure 7 below, you can see that after re-positioning the vehicle at its original configuration, the “odom” and “base_link” frames are not aligned. It is less likely that the frames would re-align since the vehicle’s overall traveling distance is increasing.

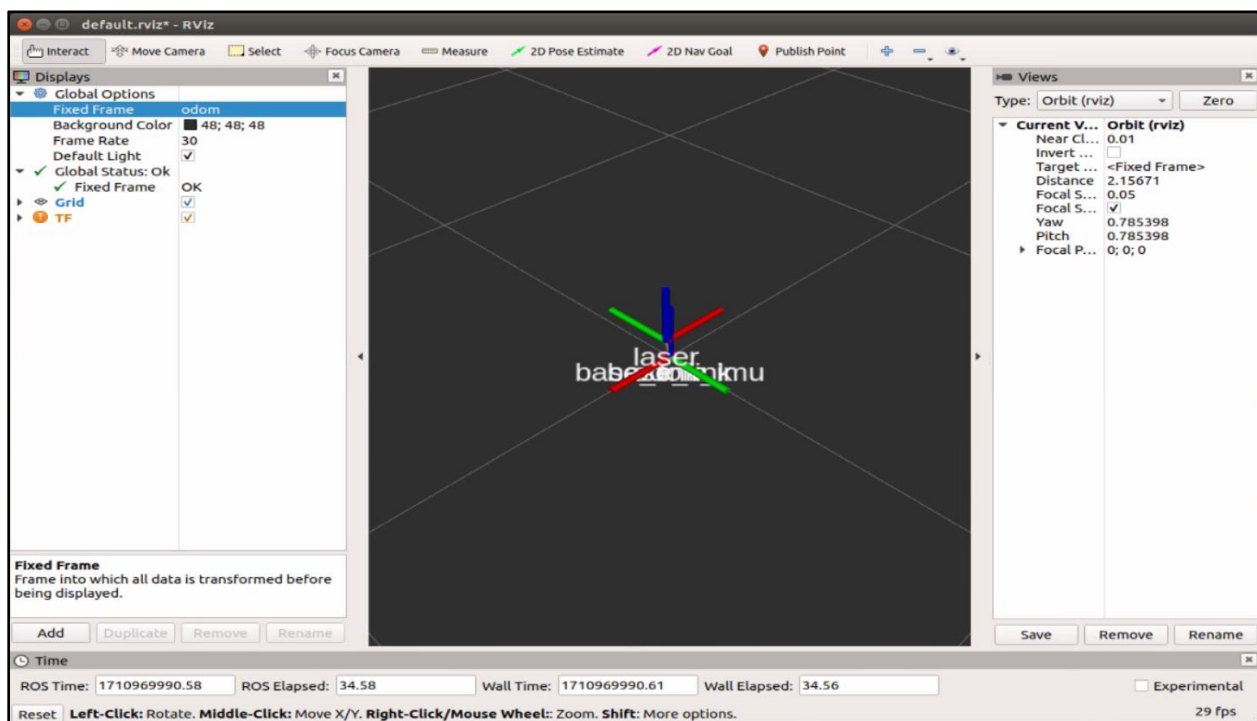


Figure 6. Aligned “odom” and “base_link” Frames before Driving the Vehicle

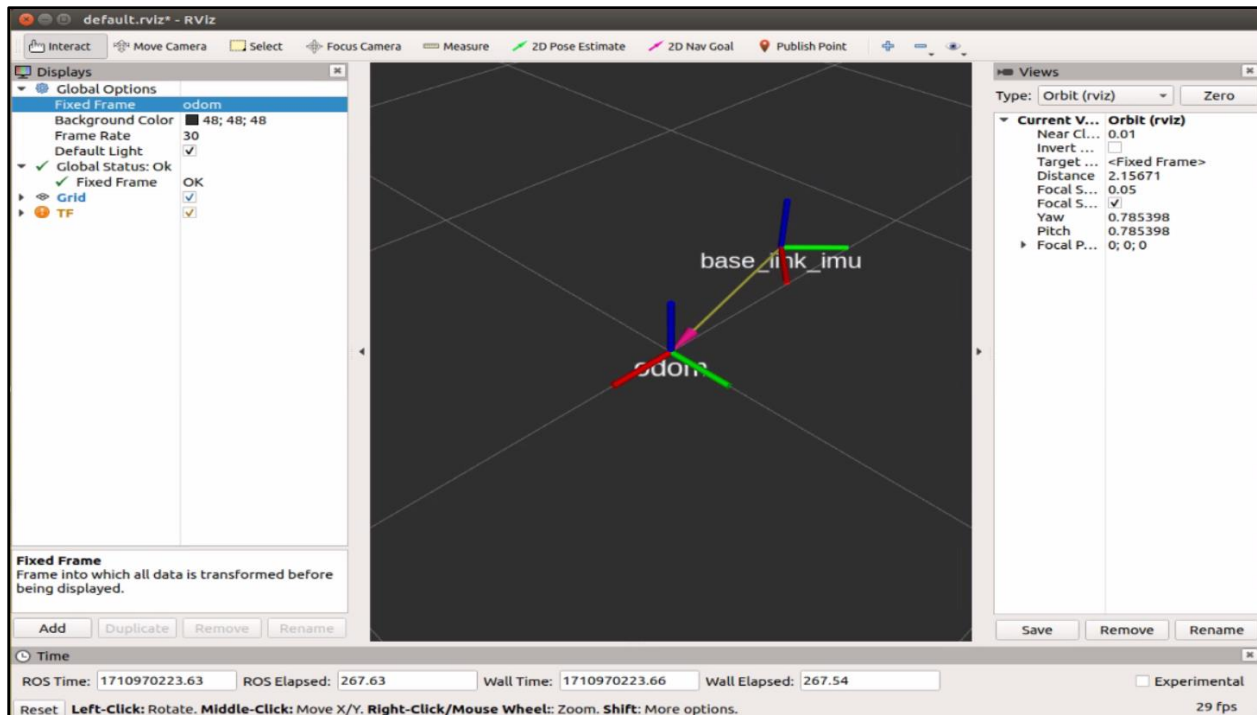


Figure 7. Position of “odom” and “base_link” Frames after Driving Vehicle to Original Configuration

Activity 13

When examining potential errors in the wheel odometry, it is clear that the alignment and slippage of the AEV's wheels play a significant role. Upon inspecting our AEV, we noticed that the back wheels are slightly misaligned, which causes the vehicle to drive at an angle even when using the joystick to drive in a straight line. Additionally, we observed that the AEV sometimes loses traction and friction with the ground when driving, which leads to errors in the wheel-based odometry. Finally, uncertainties in the encoder readings can also be a potential source of error.

Activity 14

In activity 14, we rewrote the discrete-time localization equations. To do this, we replaced the integration of the yaw angle with the associated measurement from the IMU.

Activity 15

In activity 15, we completed any missing lines of code and uncommented relevant sections in the “experiment.cpp” file in order to publish a new odometry message and a new ROS ts using the IMU yaw angle measurement.

Activity 16

Figure 8 below displays the results of vehicle localization with the “base_link_imu” frame. Imu and laser follow base_link always and go in circles. The imu orientation does not change so that it always goes in circles.

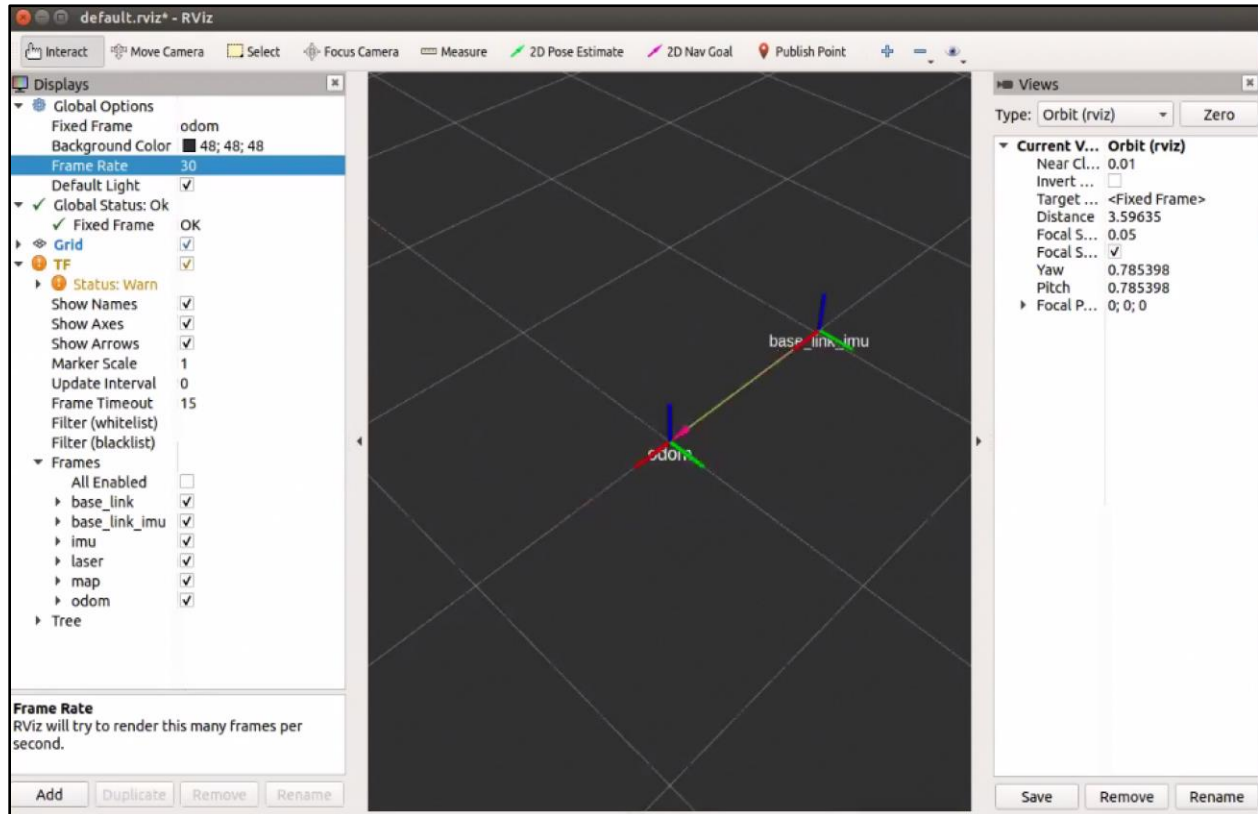


Figure 8. Vehicle Localization with the “base_link_imu” Frame

Figure 9 below displays the results of vehicle localization with the “base_link” frame.

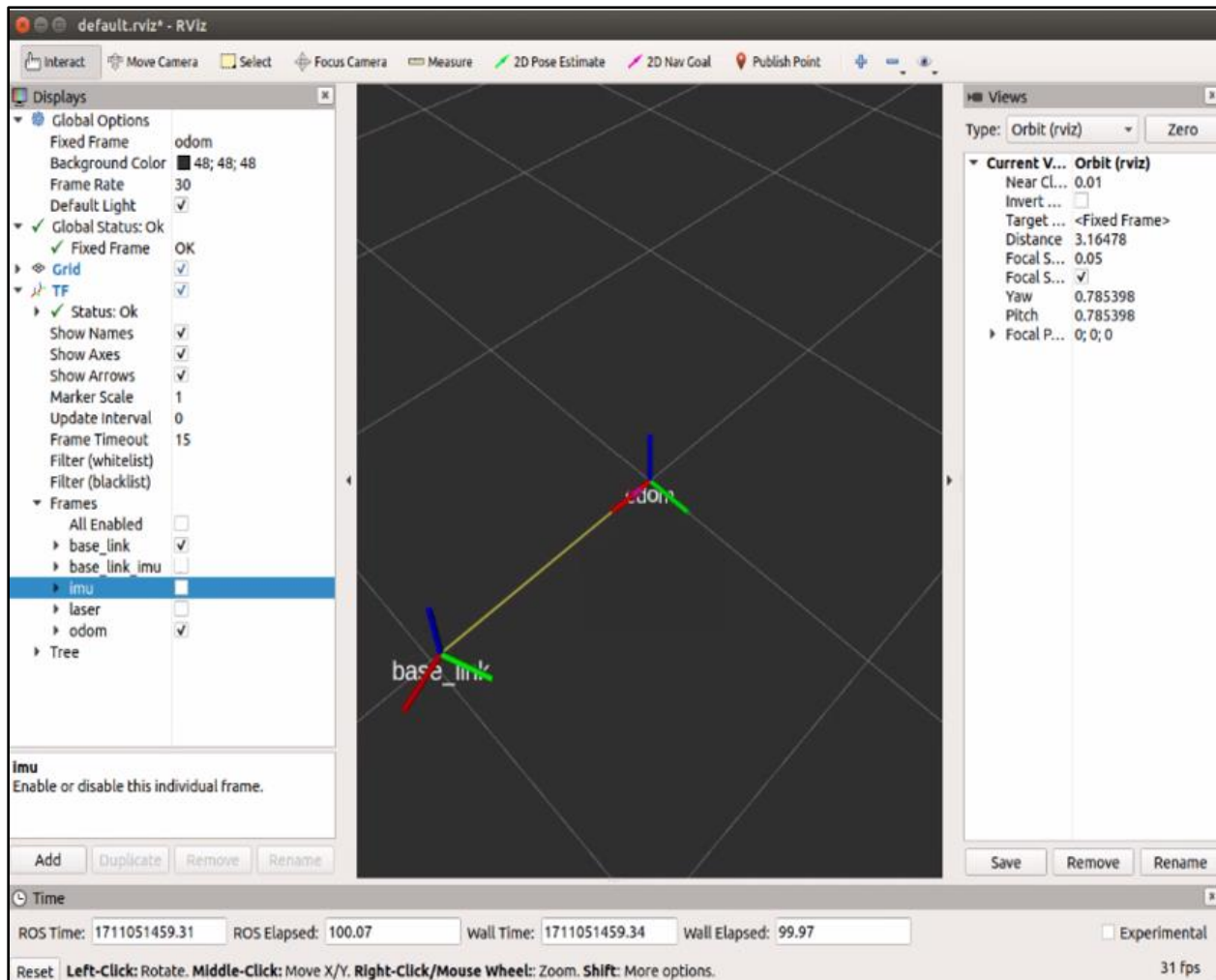


Figure 9. Vehicle Localization with the “base_link” Frame

Without using the IMU, aligning frames is easy as the odometry measures the vehicle's motion relative to the ground. However, incorporating the IMU's yaw angle measurement makes alignment more challenging as the IMU provides measurements in the Earth-North-Up frame, whereas the wheel odometry is relative to the AEV's local frame. The “experiment.cpp” file code has been designed to address the difference in the frame of reference by implementing transformations between the ENU frame of the IMU and the local frame of the wheel odometry. Precisely, it performs calculations to determine the variance in yaw angles between the IMU and the wheel odometry, which is then utilized to modify the localization data. Even with IMU data, there can be localization error accumulation due to factors such as sensor noise, IMU measurement drift, wheel odometry inaccuracies, and frame alignment discrepancies. IMU helps improve accuracy, but doesn't fully eliminate error accumulation.

Activity 17

In activity 17, we copied and built the scan tools package from “source” from its GitHub repository. After we downloaded and built the package, we uncommented the following lines of code to the “experiment.launch” file:

```
<node pkg="laser_scan_matcher" type="laser_scan_matcher_node"
  name="laser_scan_matcher_node" output="screen">
  <param name="fixed_frame" value = "odom"/>
  <param name="max_iterations" value="10"/>
  <param name="base_frame" value="base_link"/>
  <param name="use_imu" value="true"/>
  <param name="use_odom" value="true"/>
  <param name="kf_dist_linear" value="0.1"/>
  <param name="kf_dist_angular" value="0.175"/>
</node>
```

Activity 18

The roles of the parameters in the Scan Matcher algorithm are:

“use_odom”: This determines whether odometry data from the wheel encoders is used in the Scan Matcher algorithm. Odometry provides estimates of the robot's motion by tracking changes in wheel rotations.

“use_imu”: This controls whether the data from an IMU is utilized in the Scan Matcher algorithm. An IMU provides measurements of linear acceleration and angular velocity, aiding in estimating the robot's motion and orientation.

“kf_dist_linear”: This specifies the threshold distance for linear motion between consecutive scan matching iterations. It determines the minimum distance the robot must travel before initiating a new scan matching process.

“kf_dist_angular”: This parameter sets the threshold for angular motion between consecutive scan matching iterations. It determines how much angular change is needed prior to updating the position and orientation relative to the reference frame.

Activity 19

In activity 19, we located and opened the “laser_scan_matcher.cpp” file from the scan tools package. The lines of code perform mathematical transformations to update the pose in the world frame based on sensor measurements and corrections.

“`corr_ch = base_to_laser_ * corr_ch_1 * laser_to_base_;`”: This line of code essentially transforms the correction of the laser's position from the laser frame to the base frame, then applies it to correct the base's position in the base frame.

“`base_to_laser_`”: This represents the homogeneous transformation from the base frame to the laser frame.

“`corr_ch_1`”: This represents the correction of the laser's position in the laser frame.

“`laser_to_base_`”: This represents the homogeneous transformation from the laser frame to the base frame.

“`f2b_ = f2b_kf_ * corr_ch;`”: This line of code applies the correction computed in the base frame to update the pose of the robot in the world frame.

“`f2b_kf_`”: This represents the homogeneous transformation from the base frame to the world frame.

“`corr_ch`”: This represents the correction of the base's position in the base frame that were previously computed

Mathematically, these lines of code involve matrix multiplications to perform coordinate transformations between different frames of reference. By updating the robot's pose based on sensor measurements and corrections, the algorithm effectively refines the estimate of the robot's position and orientation in the world frame, improving localization accuracy.

Activity 20

In activity 20, we corrected the error in the code `laser_scan_matcher.cpp` and rebuild the scan tools package. We commented out 2 lines which are ‘`pr_ch`’ and ‘`pr_ch_1`’. Then we rewritten ‘`pr_ch`’ to `f2b_kf_.inverse()*pr_ch*f2b_;`

And ‘`pr_ch_1`’ was rewritten as `laser_to_base_*pr_ch*base_to_laser_;` by the removal of `f2b_.inverse()` and `pr_ch`.

Activity 21

Without wheel odometry and imu data, relying solely on the Scan Matcher algorithm for localization can pose challenges in accurately determining the car's movement. To achieve accurate localization, it's better to use the Scan Matcher algorithm in conjunction with wheel odometry and imu data.

We made an observation that the 'odom' axis remains stationary and fixed while moving the AEV. When we assessed the effectiveness of the Scan Matcher algorithm without utilizing wheel odometry and IMU data, it became clear that the algorithm has limitations, particularly in environments where visual cues are uniform. In such scenarios, the algorithm relies heavily on laser scan data to determine the AEV's position. While it performs well in environments with distinct features, it struggles in areas lacking prominent landmarks due to the lack of information about the AEV's motion, which could potentially rectify inaccuracies in the laser scan data.

During Activity 16, we compared the performance of the Scan Matcher algorithm in localization with and without wheel odometry and IMU data. We observed that even without the algorithm, we still received raw laser scan data from the LiDAR sensor. However, without the Scan Matcher algorithm processing this data, we were unable to accurately estimate the AEV's pose relative to the map. We lacked any contextual information regarding the AEV's location within the environment, which made it difficult to determine the AEV's position and orientation relative to the map.

Activity 22

To add a laserscan with topic scan and two different frames, namely "base_link" and "odom," it is important to examine both frames and compare them. It becomes evident that when the fixed frame is set to "odom," the odometry axis remains static and unaffected by the movement of the AEV (Autonomous Electric Vehicle). The "odom" frame is the odometry reference frame, while the laser scan data is observed from a perspective that focuses on the AEV. As the "odom" frame remains immobile, only the base_link undergoes movement. When examining the "base_link" frame, it is noted that both the "odom" and "base_link" frames are in motion as the AEV is driven. This happens because the "base_link" frame is affixed to the vehicle and moves in tandem with the vehicle's motion. Because the "odom" frame remains static in relation to the surrounding environment and serves as a reference frame, any motion of the vehicle results in the "base_link" frame moving within the "odom" frame. Since our frame of reference is based on "base_link," both frames are observed to move synchronously.

To transfer LiDAR data to the AEV frame, the raw scan data is first collected in the LiDAR's frame. However, since the LiDAR sensor is mounted on the robot, we need to align it with the AEV frame (also known as "base_link"). This is done by applying a transformation between the LiDAR's frame and the AEV frame, taking into account the position and orientation of the LiDAR sensor.

When a LiDAR data is collected by a robot, it is obtained in the robot's frame. To make sense of the data and use it for further processing, it needs to be transformed into a fixed frame, which is called "odom" in this scenario. This transformation is done by using the robot's pose relative to the fixed frame. The robot's pose can be determined through various localization techniques such as wheel odometry, IMU, or SLAM (Simultaneous Localization and Mapping).

Activity 23

In activity 23, we installed the hector SLAM on our system. To do this, we used the package manager and executed the “sudo apt-get install ros-melodic- Hector-SLAM” command. Then, we modified the “experiment.launch” file so we are able to run the hector SLAM. We commented out the section of code related to the Scan Matcher algorithm and added the following lines of code to the file:

```
<node pkg="hector_mapping" type="hector_mapping" name="hector_mapping"
output="screen">
```

```
<!-- Frame names -->
```

```
<param name="map_frame" value="odom"/>
```

```
<param name="base_frame" value="base_link" />
```

```
<param name="odom_frame" value="base_link" />
```

```
<!-- Tf use -->
```

```
<param name="use_tf_scan_transformation" value="true"/>
```

```
<param name="use_tf_pose_start_estimate" value="false"/>
```

```
<param name="pub_map_odom_transform" value="true"/>
```

```
<!-- Map size / start point -->
```

```
<param name="map_resolution" value="0.050"/>
```

```
<param name="map_size" value="$2048"/>
```

```
<param name="map_start_x" value="0.5"/>
```

```
<param name="map_start_y" value="0.5" />
```

```
<param name="map_multi_res_levels" value="2" />
```

```
<!-- Map update parameters -->
```

```
<param name="update_factor_free" value="0.4"/>
```

```
<param name="update_factor_occupied" value="0.9" />
```

```
<param name="map_update_distance_thresh" value="0.4"/>
```

```
<param name="map_update_angle_thresh" value="0.15" />
```

```
<!-- Advertising config -->
```

```
<param name="advertise_map_service" value="true"/>
```

```

<param name="scan_subscriber_queue_size" value="5"/>
<param name="scan_topic" value="scan"/>

<param name="tf_map_scanmatch_transform_frame_name"
value="scanmatcher_frame" />
</node>

```

Activity 24

In activity 24, we launched the modified “experiment.launch” file and started the SLAM algorithm. Additionally, we launched the ROS visualization tool “rviz” to view the results of the SLAM algorithm. We set the “rviz” fixed frame to “map” and added the TF and Map types displays. We selected only the “map” and “base_link” frames, and displayed the map data. We drove the vehicle around and observed how the map environment was constructed while the vehicle was simultaneously localized in the map. As we drove the vehicle around, we observed that the base_link axis is moving accordingly in the map environment.

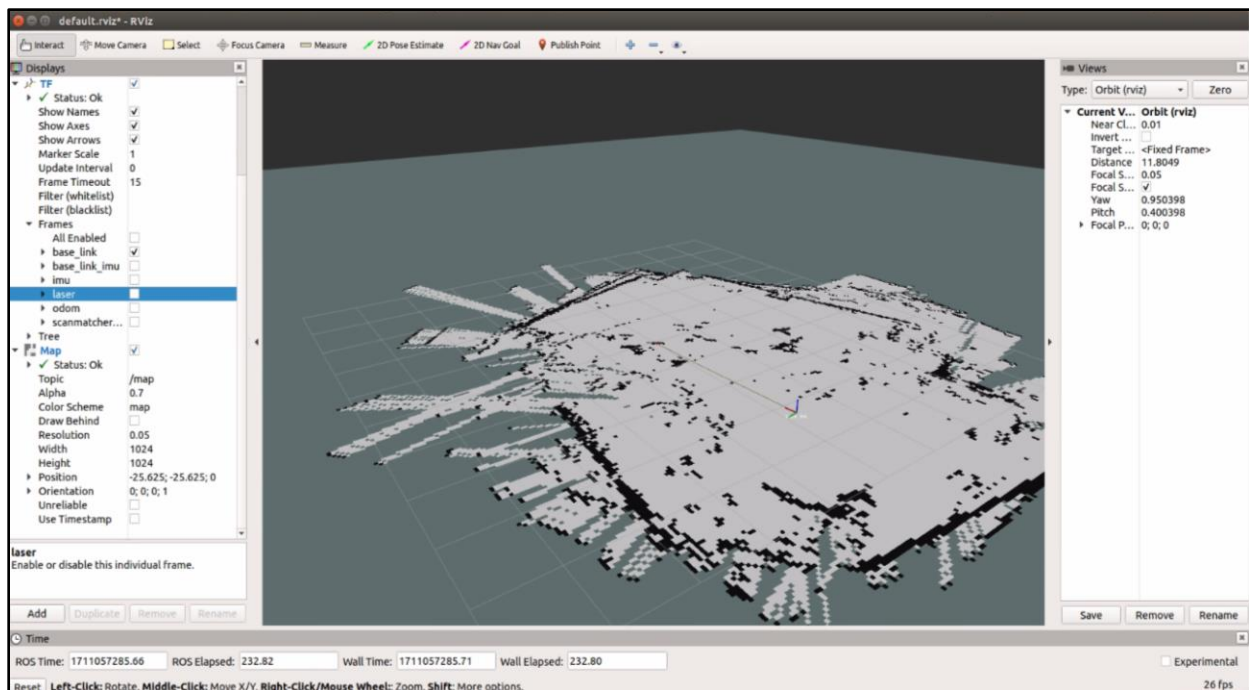


Figure 10. Constructed Map Environment

Activity 25

In activity 25, we tried to observe the potential issue of the algorithm not recognizing if you revisited a part of the environment that you have seen previously, and produces erroneous results. To observe this, we drove the vehicle around and then returned to the area that we explored. Then, we used the “`roslaunch map_server map_saver -f mymap`” command to save a copy of the construct map.

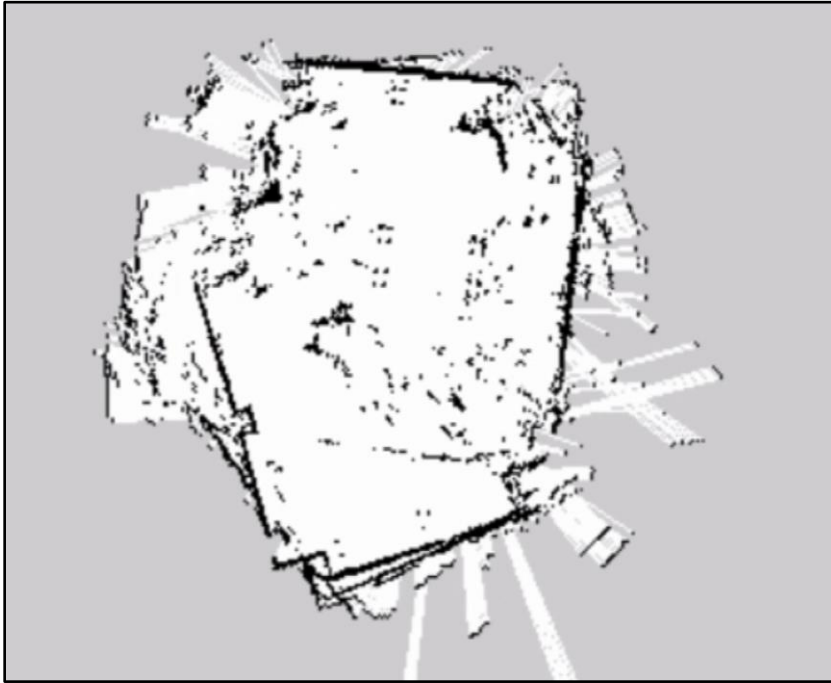


Figure 11. Construct Map